```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
# 1. import database
```

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('/content/customer_churn.db')

sql_query = """
SELECT name
FROM sqlite_master
WHERE type='table'
"""

tables = pd.read_sql(sql_query, conn)

for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

```
# print table names and column names

conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\ntable Name: {table_name}")
    # get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

#close connection
conn.close()
```

```
table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_d

table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

```
# data cleaning
```

```
df_db_customer.head()
```

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   customerid  21 non-null     object
 1   name        21 non-null     object
 2   country     18 non-null     object
 3   state       21 non-null     object
 4   gender      21 non-null     object
 5   dob         21 non-null     object
 6   interests   4 non-null      object
 7   pincode     0 non-null      object
dtypes: object(8)
memory usage: 1.4+ KB
```

```
# a.rename col - name
# b. drop columns - interest and pincode
# c. change data type - dob
# d. data standardization - gernder
# e.  fix missing value -country
```

```
# a.rename col - name

df_db_customer.rename(columns = {'name' : 'customer_name'},)
```

	customerid	customer_name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
5	0013-MHZWF	durga	None	Delhi	Women	1988-12-10 00:00:00	None	None
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	None	None
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	None	None
8	0015-UOCOJ	maya	None	Kathmandu	Women	1985-07-07 00:00:00	None	None
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	None	None
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	None	None
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	None	None
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01 00:00:00	None	None
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	None	None
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	None	None
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	None	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	None	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	None	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	None	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	None	None

```
# b.drop columns - interest and pincode
```

```
# df_db_customer.drop(df_db_customer.columns[-2:])
df_db_customer.columns[-2:]
```

```
Index(['interests', 'pincode'], dtype='object')
```

```
df_db_customer.drop(columns=['interests', 'pincode'],)
```

	customerid	name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	None	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	None	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

```
# c.change data type - dob

pd.to_datetime(df_db_customer['dob'])
```

	dob
0	1982-04-12
1	1995-11-23
2	1978-02-15
3	2001-08-30
4	1990-05-05
5	1988-12-10
6	1976-09-21
7	1999-03-14
8	1985-07-07
9	1993-10-29
10	1997-01-22
11	1981-06-18
12	2004-12-01
13	1992-04-25
14	1979-11-11
15	1986-02-28
16	1994-08-19
17	2000-09-02
18	1983-12-30
19	1991-05-14
20	1977-10-06

dtype: datetime64[ns]

```
# d. data standardiztion - gernder

# df_db_customer['gender'].unique()
df_db_customer['gender'].replace({'men' : 'male', 'women' : 'female'})
```

	gender
0	Male
1	Male
2	Female
3	Male
4	Female
5	Women
6	Female
7	Male
8	Women
9	Male
10	Men
11	Female
12	Female
13	Female
14	Men
15	Female
16	Female
17	Female
18	Male
19	Men
20	Women

dtype: object

```
# e. fix missing value - country

df_db_customer[df_db_customer ['country'].isna()]
```

	customerid	name	country	state	gender	dob	interests	pincode
5	0013-MHZWF	durga	None	Delhi	Women	1988-12-10 00:00:00	None	None
8	0015-UOCOJ	maya	None	Kathmandu	Women	1985-07-07 00:00:00	None	None
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01 00:00:00	None	None

```
df_db_customer['country'].fillna('nahi pata')
```

	country
0	India
1	India
2	India
3	India
4	India
5	nahi pata
6	India
7	India
8	nahi pata
9	Nepal
10	India
11	India
12	nahi pata
13	India
14	India
15	India
16	India
17	India
18	India
19	India
20	India

dtype: object

```
df_db_customer[['country','state']]
```

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	None	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	None	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	None	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

```
#df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()
```

```
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()

df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))
```

	country
0	India
1	India
2	India
3	India
4	India
5	India
6	India
7	India
8	Nepal
9	Nepal
10	India
11	India
12	India
13	India
14	India
15	India
16	India
17	India
18	India
19	India
20	India

dtype: object

```
df_db_customer[df_db_customer['country'].isna()]
```

	customerid	name	country	state	gender	dob	interests	pincode
5	0013-MHZWF	durga	None	Delhi	Women	1988-12-10 00:00:00	None	None
8	0015-UOCOJ	maya	None	Kathmandu	Women	1985-07-07 00:00:00	None	None
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01 00:00:00	None	None

```
df_db_subscription.head()
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancel
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switch
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	

Next steps: Generate code with `df_db_subscription` New interactive sheet

```
print(df_db_subscription.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_d
```

```
print(df_db_subscription.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_d
```

```
df_db_subscription.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score'],
      dtype='object')
```

```
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type       21 non-null     object
 3   renewal_date            21 non-null     object
 4   plan_type               21 non-null     object
 5   contract_type           21 non-null     object
 6   cancellation_date       6 non-null      object
 7   cancellation_reason     6 non-null      object
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

```
date_col = ['subscription_start_date',
            'renewal_date',
            'cancellation_date']

df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type       21 non-null     object
 3   renewal_date            21 non-null     object
 4   plan_type               21 non-null     object
 5   contract_type           21 non-null     object
 6   cancellation_date       6 non-null      object
 7   cancellation_reason     6 non-null      object
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

```
df_db_support.head()
```

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      object
```

```
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

```
df_db_support.drop(columns=['col_1','comment'],inplace=True)
```

```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64 
dtypes: int64(1), object(3)
memory usage: 420.0+ bytes
```

```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

# df_db_support.info()
```

```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype         
---  ------          --------------  -----         
 0   customerid      9 non-null      object        
 1   complaint_date  9 non-null      datetime64[ns]
 2   escalations     9 non-null      object        
 3   csat_score      9 non-null      int64         
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 420.0+ bytes
```

```
Start coding or generate with AI.
```

```
# 3.  Feature Engineering and data Analysis
```

```
np.where(df_db_subscription['cancellation_date'].notna(),1,0)
```

```
array([0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0])
```

```
df_db_subscription.head()
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancel
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switch
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	

Next steps: Generate code with df_db_subscription | New interactive sheet

```
df_db_subscription.merge(df_db_customer , on = 'customerid', how= 'left')
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cance
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switc
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual	None	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Nc
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual	None	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly	None	
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual	None	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual	None	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	Annual	None	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	Monthly	2024-10-31	Switc
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	Annual	None	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	Monthly	None	
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	None	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	None	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Fo
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	None	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	None	

```
(df_db_subscription
            .merge(df_db_customer,on = 'customerid', how = 'left')
            .merge(df_db_customer,on = 'customerid', how = 'left'))
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cance
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switc
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual	None	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Nc
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual	None	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly	None	
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual	None	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual	None	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	Annual	None	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	Monthly	2024-10-31	Switc
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	Annual	None	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	Monthly	None	
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	None	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	None	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Fo
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	None	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	None	

21 rows × 25 columns

```
df.shape
```

```
(9, 4)
```

```
df_db_subscription.shape
```

```
(21, 11)
```

```
df.head()
```

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30

Next steps: Generate code with df | New interactive sheet

```
df_db_subscription['customerid'].nunique()
```

```
21
```

```
df_db_customer['customerid'].nunique()
```

```
21
```

```
df_db_support['customerid'].nunique()
```

```
7
```

```
df_db_support['customerid'].size
```

```
9
```

```
df_db_support
```

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

Next steps: Generate code with df | New interactive sheet

```
df_db_support.groupby('customerid')['customerid'].transform('count')
```

	customerid
0	2
1	2
2	1
3	1
4	1
5	1
6	1
7	2
8	2

dtype: int64

```
df['cancellation_date'] = pd.to_datetime(df['cancellation_date']) # Ensure cancellation_date is datetime
df['churn_flag'] = np.where(df['cancellation_date'].notna(), 1, 0)
churn_by_subscription_type = df.groupby('subscription_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn
print(churn_by_subscription_type)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'cancellation_date'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                    2 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'cancellation_date'
```

Next steps: Explain error

```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid', keep = 'last')
```

```
df_db_support['customerid'].size
```

```
7
```

```
# merge df
df = (df_db_subscription
      .merge(df_db_customer,on = 'customerid', how = 'left')
      .merge(df_db_support,on = 'customerid',how = 'left'))
```

```
df.shape
```

```
(21, 21)
```

```
df.to_csv('exported_churn_data.csv', index=False)
```

```
# Data Analysis
```

```
# 1. Churn Rate
```

```
df.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
       'country', 'state', 'gender', 'dob', 'interests', 'pincode',
       'complaint_date', 'escalations', 'csat_score'],
      dtype='object')
```

```
df['churn_flag'] = np.where(df['cancellation_date'].notna(), 1, 0)
churn_rate = df['churn_flag'].mean()*100
print("churn rate = " , round(churn_rate,2),"%")
```

```
churn rate =  28.57 %
```

```
# 2 Retenion Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2),"%")
```

```
Retention Rate =  71.43 %
```

```
df.head(2
        )
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancel
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switch

2 rows × 22 columns

```
# 3. churn by plan type

churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate'))
print(churn_by_plan)
```

```
  plan_type  churn_rate
0     Basic       60.00
1   Premium       14.29
2  Standard       22.22
```

```
# 4 a. churn by
df = pd.read_csv("/content/exported_churn_data.csv")

# State-wise Revenue Sum और User Count
result = df.groupby('state').agg(
    Total_Revenue=('monthly_charges', 'sum'),
    User_Count=('customerid', 'count')
).reset_index()

print(result)
# b.  churn by subscription type
```

```
           state  Total_Revenue  User_Count
0          Delhi          52.96           4
1      Karnataka          20.98           2
2      Kathmandu          20.98           2
3    Maharashtra          50.97           3
4      Meghalaya          42.97           3
5       Nagaland          22.99           1
6      Rajasthan          36.98           2
7      Telangana          30.98           2
8  Uttar Pradesh         115.98           2
```

```
# b.  churn by subscription type
```

```
print(churn_by_subscription_type)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_843/2987492461.py in <cell line: 0>()
----> 1 print(churn_by_subscription_type)

NameError: name 'churn_by_subscription_type' is not defined
```

Next steps: Explain error

```
# 5.  Arpu - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

Start coding or generate with AI.

```
#6 avg Customer Tenure
#count of days users has used our service : canncellation else currrent date
Today = pd.Timestamp.today()
df['cancellation_date'] = pd.to_datetime(df['cancellation_date'])
df['subscription_start_date'] = pd.to_datetime(df['subscription_start_date'])
df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (Today - df['subscription_start_date']).dt.days
)
avg_tenure  = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1549 0
```

```
# Revenue at risk - revenue Last from churnrd users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk(Rs lakhs)=", revenue_at_risk)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'churn_flag'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                  2 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'churn_flag'
```

Next steps: Explain error

```
df.head(2)
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancel
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switch

2 rows × 22 columns

```
# 8 Escalation rate
escalations_rate = (df['escalations']=='Y').mean()*100
```

```
#9 . Avg complaint per user
# Calculate the number of complaints per customer from df_db_support
complaints_per_customer = df_db_support.groupby('customerid').size().reset_index(name='complaint_count')

# Merge this complaint count into the main df
df = df.merge(complaints_per_customer, on='customerid', how='left')

# Fill NaN values (for customers with no complaints) with 0
df['complaint_count'] = df['complaint_count'].fillna(0)

# Calculate the average complaint per user
avg_complaints_per_user = df['complaint_count'].mean()

print(f"Average complaints per user: {avg_complaints_per_user:.2f}")
```

```
Average complaints per user: 0.33
```

```
import numpy as np
import pandas as pd

# Ensure 'cancellation_date' is in datetime format
df['cancellation_date'] = pd.to_datetime(df['cancellation_date'])

# Re-create 'churn_flag' as it was lost after reloading the DataFrame from CSV
df['churn_flag'] = np.where(df['cancellation_date'].notna(), 1, 0)

#10. correlation Esclation vs churn
corr_df = df[['customerid','escalations','churn_flag']].dropna()

# Convert 'escalations' to numeric (1 for 'Y', 0 for 'N')
corr_df['escalations_numeric'] = np.where(corr_df['escalations'] == 'Y', 1, 0)

correlation = corr_df['escalations_numeric'].corr(corr_df['churn_flag'])
print(f"Correlation between Escalations and Churn: {correlation:.2f}")
```

```
Correlation between Escalations and Churn: 0.47
```

```
#11.  churn risk - create a column using existing col

conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >=50) & (df['churn_score']< 70),
    (df['csat_score'] >= 70)
]
choices = ['low','med','high']
df['churn_risk'] = np.select(conditions, choices,default='unkown')
```

```
# 4 visualization using matplotilb
```

```
df_visual = df.copy()
```

```
df_visual.shape
```

```
(21, 25)
```

```
df.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
       'country', 'state', 'gender', 'dob', 'interests', 'pincode',
```

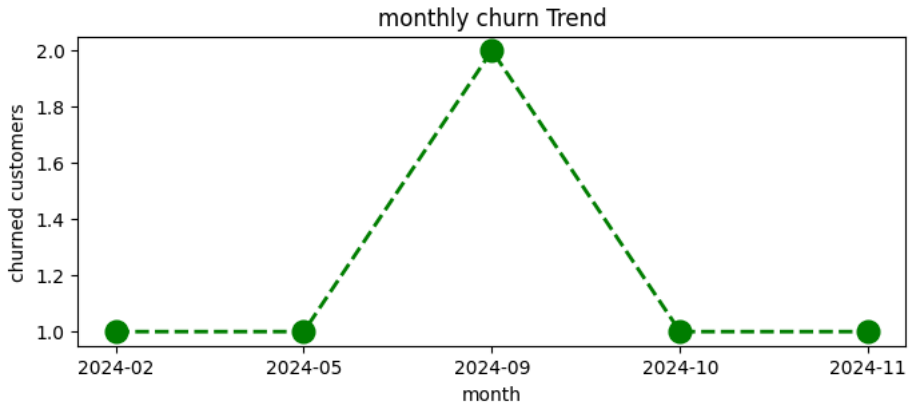
```
       'complaint_date', 'escalations', 'csat_score', 'tenure_days',
       'complaint_count', 'churn_flag', 'churn_risk'],
      dtype='object')
```

```
from matplotlib.lines import lineStyles
# 4.1 Monthly churn trend (TIME SERICEE KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(8,3))

plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green' , marker='o',linestyle='dashed', linewidth=2,mark
plt.title('monthly churn Trend')
plt.xlabel('month')
plt.ylabel('churned customers')
plt.show()
```




```
import pandas as pd
import numpy as np

#4.2 churn by plan type
# Ensure df is defined and processed before creating df_visual
df = pd.read_csv("/content/exported_churn_data.csv") # Reload df
df['cancellation_date'] = pd.to_datetime(df['cancellation_date']) # Ensure datetime format
df['churn_flag'] = np.where(df['cancellation_date'].notna(), 1, 0) # Recreate churn_flag

df_visual = df.copy() # Now df should be defined
churn_by_plan_type = df_visual.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name='churn_rate')
print(churn_by_plan_type)
```

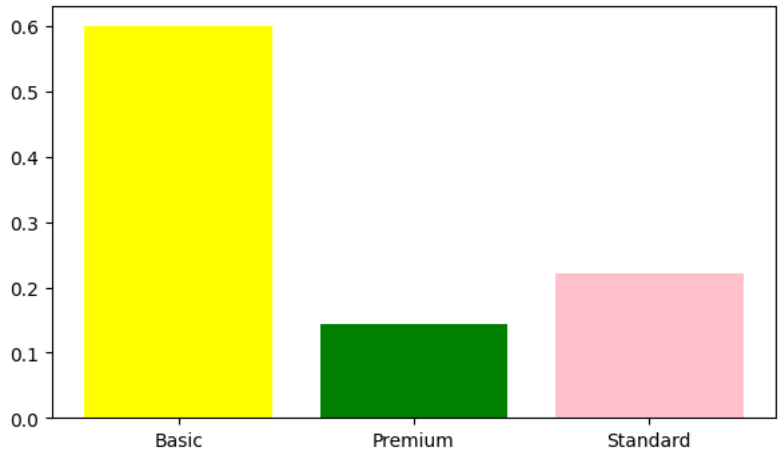
```
  plan_type  churn_rate
0     Basic       60.00
1   Premium       14.29
2  Standard       22.22
```

```
# 4.2 churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = ['yellow', 'green', 'pink']

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index,churn_plan.values,color = colors)
plt.show()
```

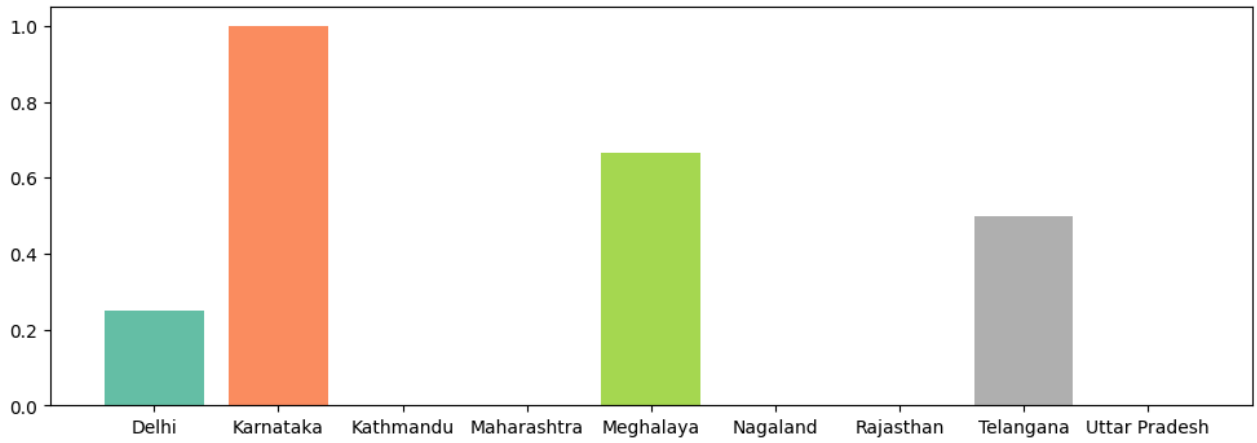



```
# 4.3  churn by plan type
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

#colors = ['yellow', 'green', 'pink']
colors = plt.colormaps.get_cmap('Set2')(np.linspace(0,1, len(churn_plan)))

plt.figure(figsize=(12,4))

plt.bar(churn_plan.index,churn_plan.values,color = colors)
plt.show()
```




```
# visulaization using seaborn
```

```
# encoding - (correlation  str to numeric so that we can find carr betwwen frature )
df_visual.columns
```

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
       'country', 'state', 'gender', 'dob', 'interests', 'pincode',
       'complaint_date', 'escalations', 'csat_score', 'churn_flag'],
      dtype='object')
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >=50) & (df['churn_score']< 70),
    (df['csat_score'] >= 70)
]
choices = ['low','med','high']
df['churn_risk'] = np.select(conditions, choices,default='unkown')

df_encoded = df[[
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'name',
       'country', 'state', 'gender', 'dob', 'interests', 'pincode',
       'complaint_date', 'escalations', 'csat_score', 'churn_flag', 'churn_risk']].head()
```

```
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']
for col in categorical_cols:
  df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
# Heatmap(correlation matrix)

# Make a copy to avoid modifying the original df_encoded (which might be only 5 rows)
df_for_corr = df_encoded.copy()

# Convert 'escalations' from 'Y'/'N' to 1/0
df_for_corr['escalations'] = df_for_corr['escalations'].map({'Y': 1, 'N': 0}).fillna(-1)

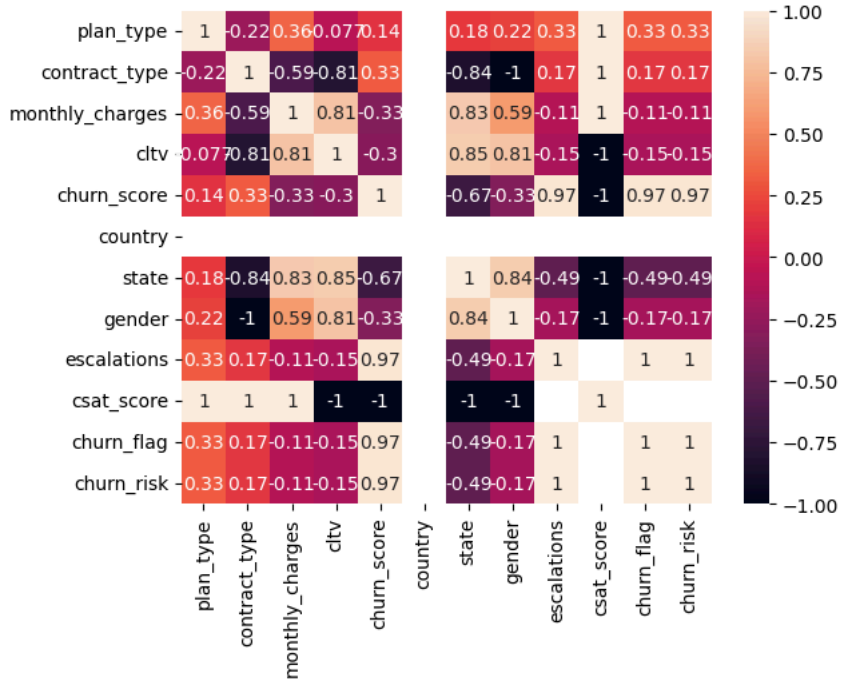
# List of columns to drop for correlation calculation (dates, strings, IDs)
columns_to_drop = [
    'renewal_date', 'cancellation_date', 'cancellation_reason', 'name',
    'dob', 'interests', 'pincode', 'complaint_date'
]

# Drop the non-numeric columns that are not meant for encoding or are dates/text
df_for_corr = df_for_corr.drop(columns=[col for col in columns_to_drop if col in df_for_corr.columns])

# Re-encode other categorical columns if they are still present as strings
categorical_cols_to_encode_further = ['country', 'state', 'gender']
for col in categorical_cols_to_encode_further:
    if col in df_for_corr.columns:
        df_for_corr[col] = df_for_corr[col].astype('category').cat.codes

sns.heatmap(df_for_corr.corr(),annot=True)
```

```
<Axes: >
```




```
df_encoded.head()
```

	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score	nam
0	2025-03-15	2	0	NaT	NaN	13.99	627	12	kesha
1	2024-08-01	1	0	2024-09-10	Switched to competitor	12.99	1150	91	ragha
2	2025-11-20	0	1	NaT	NaN	6.99	210	34	lalit
3	2025-05-10	1	0	NaT	NaN	22.99	1725	8	mohar
4	2024-01-05	2	1	2024-02-28	Too expensive	13.99	195	88	mir

Next steps: Generate code with df_encoded | New interactive sheet

```
df.head(2)
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancel
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switch

2 rows × 23 columns

```
from typing import assert_type
from pandas.core.api import Categorical
#correct method of enoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type','churn_score', 'churn risk', 'escalations']]

order_mappings = {
    'plan_type' : ['Basic', 'standard', 'premium'],
    'contract_type' : ['monthly', 'Annual'],
    'churn_risk' : ['low', 'med' 'high']

    for col, order in order_mappings.items():
      df_encoded[col] = pd.Categorical(df_encoded[col].astype('category')=order, ordered=True).codes
}
```

```
  File "/tmp/ipykernel_843/3498918455.py", line 11
    for col, order in order_mappings.items():
    ^
SyntaxError: invalid syntax
```

Next steps: Explain error

```
df_visual[['plan_type', 'contract_type', 'churn_flag','churn_score', 'churn_risk','escalations']].head()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_843/1042213895.py in <cell line: 0>()
----> 1 df_visual[['plan_type', 'contract_type', 'churn_flag','churn_score', 'churn_risk','escalations']].head()

                                 2 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in _raise_if_missing(self, key, indexer, axis_name)
   6250
   6251             not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6252             raise KeyError(f"{not_found} not in index")
   6253
   6254     @overload

KeyError: "['churn_risk'] not in index"
```

Next steps: Explain error

```
df_encoded.head()
```

	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score	nam
0	2025-03-15	2	0	NaT	NaN	13.99	627	12	kesha
1	2024-08-01	1	0	2024-09-10	Switched to competitor	12.99	1150	91	ragha
2	2025-11-20	0	1	NaT	NaN	6.99	210	34	lalit
3	2025-05-10	1	0	NaT	NaN	22.99	1725	8	mohar
4	2024-01-05	2	1	2024-02-28	Too expensive	13.99	195	88	mir

Next steps: Generate code with `df_encoded` | New interactive sheet

```
# Heatmap (correlation matrix)

# Make a copy to process 'escalations' for correlation
df_for_heatmap = df_encoded.copy()

# Convert 'escalations' to numeric (1 for 'Y', 0 for 'N')
# Fill NaN values with 0, assuming NaN means no escalation was reported
df_for_heatmap['escalations'] = df_for_heatmap['escalations'].map({'Y': 1, 'N': 0}).fillna(0)

# Now calculate correlation and plot heatmap
sns.heatmap(df_for_heatmap.corr(), annot=True)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_843/2832628836.py in <cell line: 0>()
      9
     10 # Now calculate correlation and plot heatmap
---> 11 sns.heatmap(df_for_heatmap.corr(), annot=True)
```

3 frames

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.